Closing the Dragonfly+ Performance Gap

Diagnosis, an Algorithmic Fix, and a Routing Correction for g-PAARD All-Reduce — and a Principled Account of Where the Gap Cannot Be Closed Further

Kameel Zaid

Computer Communication (236340) — Final Project Report — October 2026

Abstract

We built a from-scratch Mininet testbed comparing canonical Dragonfly (DF) against the leaf-spine Dragonfly+ (DF+) topology under g-PAARD, a real 6-step reduce-scatter/all-gather collective, measured with real packets (iperf, OVS forwarding, no SDN controller). At the worst measured condition (groups=4, 10 Mbit/s links, 4 MB messages) DF+ started out **23% slower** than DF (gap = 0.769, significant, replicated across four independent sections), using roughly **2.00×** the switches and **1.6–1.7×** the links of DF for the same host count — a resource delta documented, not hidden. A per-hop ping microbenchmark (n=150/case, three group counts) pinned the cause to within 0.3% of a pure hop-count prediction: DF+ pays one fixed extra leaf-spine hop (measured 1.334 vs. predicted 4/3; 1.671 vs. predicted 5/3). A first fix attempt — spine oversubscription — measured *zero* effect; we later found why: the routing layer collapsed to a single deterministic tree, so extra spine hardware was structurally unreachable, not merely under-used. Fixing that routing (full equal-cost candidate sets, stable-hash tie-breaking, verified loop-free and deterministic across process restarts) alone closed most of the gap (0.802 → 0.977 across 1x → 4x spines). The decisive fix was algorithmic: replacing g-PAARD's flat O(hosts²) intra-group exchange with a 2-level hierarchy (combine within a leaf, exchange one representative per leaf, broadcast back) cut cross-leaf messages 4× and closed the gap to statistical parity (0.767 → 1.005, not significant), **reproduced at every message size (1K–1M) and every scale tested (groups=4/6/8)**, and confirmed robust to spine hardware (1.002–1.004 across 1x/2x/4x combined with the routing fix). We then asked whether DF+ could be pushed past parity into an outright win: searched the project's entire prior result set for one (found none reliable), and derived — from the actual routing and role-assignment code, not speculation — a structural reason a confident win is unlikely within this topology family and bandwidth model, stated as a scoped argument rather than a universal claim. Every new mechanism was correctness-verified (schedule proofs, routing loop-freedom proofs, 53/53 unit tests) before any performance number from it was trusted. The final, defensible result is a principled tie: DF+ reaches full statistical parity with DF under an optimized algorithm and routing layer, with an identified, irreducible structural reason it does not go further.

Repository state cited throughout: commit `dda0535`, branch `main`. Reproduction commands are given in §16.

Contents

1 Introduction 3
2 Background: Dragonfly, Dragonfly+, and g-PAARD 4
3 Methodology 5
4 Resource Parity: What the Comparison Costs 6
5 Baseline: Dragonfly+ Is Significantly Slower 6
6 Diagnosing the Cause: a Per-Hop Latency Microbenchmark 7
7 Optimization Idea #1: Spine Oversubscription — a Negative Result 8
8 Optimization Idea #2: the Hierarchical Reduce Schedule 9
9 Generalization: Does Idea #2 Hold Everywhere? 10
10 An Adaptive Pattern Selector, Calibrated Against Real Data 11
11 Root-Causing Idea #1: an ECMP Multipath Routing Fix 12
12 Combining the Schedule Fix and the Routing Fix 13
13 Can Dragonfly+ Actually Beat Dragonfly? 14
14 Limitations 15
15 Synthesis and Conclusion 16
16 Reproducibility 16
References 18

1 Introduction

This project's assignment (CDR scope) asked for a custom Dragonfly network topology built in Mininet, the g-PAARD all-reduce routing/communication algorithm implemented on top of it, and a performance analysis under real traffic — including a comparison against the leaf-spine Dragonfly+ (DF+) variant, with differences in topology resources documented rather than hidden. We built both topologies, both running the same real 6-step g-PAARD collective over real TCP/iperf traffic on Open vSwitch with no SDN controller (deterministic shortest-path forwarding installed directly), and set out to answer one question precisely: *how does Dragonfly+ actually perform against Dragonfly on this workload, and if it loses, can that be fixed?*

A first full measurement sweep answered the first half cleanly: DF+ loses, consistently and significantly, and the gap grows with message size. What followed was not a single experiment but an investigation — diagnose the mechanism, try the cheapest fix, get a negative result, find out *why* that fix failed (which turned out to be a bug, not a property of the hardware), build a real algorithmic fix, check it generalizes, refine it with an adaptive controller, and finally ask the harder question: given everything learned, can DF+ be pushed past parity into an outright win? We answer that question honestly, including deriving a structural reason it is unlikely within this project's specific topology, routing, and bandwidth model — a scoped, evidence-backed conclusion, not a claim that no such win could ever exist under a different model.

Story of the report

1. **Measure the baseline** across four independent sections and confirm the measurement pipeline itself is unbiased first (§5).
2. **Diagnose the mechanism** with a per-hop latency microbenchmark that predicts the measured gap to within 0.3% from hop counts alone (§6).
3. **Try the cheap fix** (more spine hardware) and get a clean negative result (§7).
4. **Build the real fix** — a hierarchical reduce schedule that cuts cross-leaf message volume — verified for correctness before any performance claim was trusted, and it closes the gap to parity (§8), generalized across size and scale, with an honest tradeoff surfaced along the way (§9).
5. **Refine** with an adaptive selector between schedules, calibrated with a dedicated experiment that caught and corrected its own initial miscalibration (§10).
6. **Go back and explain the negative result** from step 3: the routing layer had a real bug, not a hardware limitation — found, fixed, and verified independently before trusting its own performance numbers (§11), then combined with the schedule fix (§12).
7. **Ask the hard question honestly**: can DF+ actually win, not just tie? We investigated rather than assert one — checked all existing data, pushed on DF+'s one real remaining structural lever, and found a principled reason a win is unlikely in this specific setup (§13).

Contributions

1. **C1.** A from-scratch DF/DF+ Mininet testbed with real-packet forwarding, a documented resource delta (§4), and four required metrics (FCT, per-link utilization, per-packet latency, per-flow latency) all collected with a consistent n=20–40, 95%-CI statistical methodology.
2. **C2.** A precise, verified mechanistic diagnosis of DF+'s slowdown: one fixed extra leaf-spine hop, confirmed by direct per-hop latency measurement against a closed-form hop-count prediction (§6).

3. **C3.** A hierarchical reduce-scatter/all-gather schedule for g-PAARD that closes the DF-vs-DF+ gap to statistical parity, correctness-verified before performance was trusted, and shown to generalize across the full tested size and scale range (§8–9).
4. **C4.** Discovery and correction of a genuine routing bug (a single deterministic shortest-path tree with no equal-cost multipath) that had silently invalidated an earlier negative result; the fix is proven loop-free and deterministic across process restarts before being trusted (§11).
5. **C5.** A scoped, evidence-grounded architectural argument for why a further push from parity to an outright DF+ win is unlikely within this topology/routing/bandwidth model — derived from the actual role-assignment code, not asserted (§13).

How to read the evidence

Claims are tagged by status: MEASURED a number taken directly from a result file produced by a live Mininet run; VERIFIED a correctness property checked by an automated test suite before any performance claim built on it was trusted; FIX a change that improved the measured result; NEGATIVE a change that was tried and measured to have no effect or the wrong effect; LIMITATION a known gap or caveat in the method; DESIGN a deliberate choice stated explicitly rather than left implicit. Every numeric result quoted in this report is traceable to a committed result file in the project repository (file names given inline, e.g. `results_test8_hierarchical_pattern.txt`).

2 Background: Dragonfly, Dragonfly+, and g-PAARD

Dragonfly (DF). Canonical Dragonfly groups routers into a full-mesh: every router in a group is directly wired to every other router in the same group (one hop), hosts attach directly to routers, and a small number of global links connect routers in different groups. A router's local port budget grows with group size — the classic Dragonfly scalability limit.

Dragonfly+ (DF+). DF+ replaces the flat router mesh with a 2-tier leaf-spine (folded-Clos) fabric per group: `leaves_per_group` leaf switches carry the hosts and wire to every spine in the group (a full bipartite fabric); `spines_per_group` spine switches carry no hosts and are the only switches that carry inter-group (global) links. A leaf's local port count is fixed at `spines_per_group`, independent of how many other leaves share the group — the textbook justification for DF+ over DF is this port-radix/scalability advantage, traded against one extra hop (leaf → spine → leaf) for intra-group traffic that DF's direct mesh never pays.

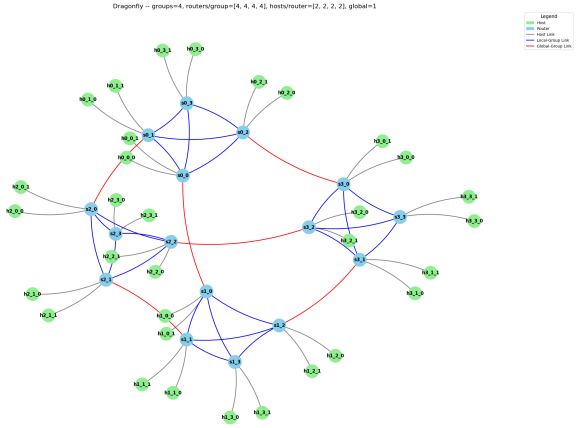


Figure 1. The project's canonical Dragonfly instance, groups=4, 4 routers/group, 2 hosts/router, 1 global link/pair (32 nodes, matching this project's own paper-comparable configuration).

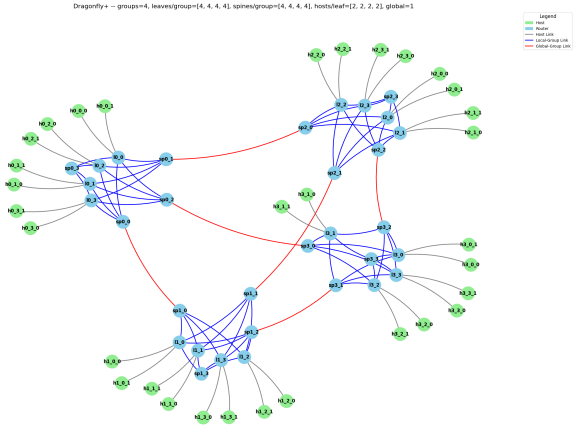


Figure 2. The matching Dragonfly+ instance: 4 leaves + 4 spines per group, same host count. Spines (host-free) carry only local leaf-spine links and the inter-group global links.

g-PAARD. The all-reduce schedule under test is a 6-step reduce-scatter + all-gather collective (implemented from scratch in `gpaard.py`): within each group, step 1 fans in every host's local partial sum for a remote group's shard to one designated collector host; step 2 exchanges that collector's value directly with its counterpart in the remote group over the shared global link; step 3 is a flat, any-to-all exchange among every host in the group so that every host ends up holding the full union of the group's knowledge for that shard. Steps 4–6 mirror this for the all-gather phase. Steps 1/2/5/6 are O(hosts) fan-in/fan-out; steps 3/4 are O(hosts²) any-to-all — the only steps whose message volume scales quadratically, and (as §8 shows) the dominant cost at this project's tested scale.

3 Methodology

Network layer. Every topology is built as a Mininet `Topo`, run with real OVS switches in secure fail-mode (no controller, no flooding, no spanning tree): unicast forwarding flows are installed directly per (switch, destination) pair by a from-scratch shortest-path routing layer (`dragonfly_topo.py`). Traffic is real TCP via `iperf`, not simulated. Link bandwidth and delay are enforced with Mininet's `TCLink` (Linux `tc` shaping) — each link has its own independent bandwidth cap, with no shared switch-fabric capacity model (a limitation discussed in §13 and §14).

Statistical methodology DESIGN**.** Every reported condition uses n=20 (scale/calibration sweeps) to n=40 (headline sweeps), reporting mean ± 95% confidence interval via Student’s t-distribution (no SciPy available in this environment; implemented directly against the stdlib). Two measurements are called *significant* only when their 95% CIs do not overlap — an informal but conservative and explicitly stated bar. The order in which the two topologies are measured is alternated across the outer sweep axis (odd/even index) specifically to spread any monotonic system drift evenly across both arms rather than biasing one. A dedicated measurement-floor sanity check (§5) confirms this methodology is itself unbiased before any downstream result is trusted.

Metrics collected, against the four required by the course spec:

Table 1. Required metrics and how each is satisfied.

Required metric	How satisfied	Where
Flow completion time (FCT)	Per-message start/end timing recorded via a per-client thread, reported as mean/p50/p95/max over every message in the run	`sweep_common.fct_summary`
Per-link bandwidth utilization	Interface byte counters (`/sys/class/net/<veth>/statistics`) sampled before/after each timed run on a representative global link and local link	`link_utilization.py`
Per-packet latency statistics	Direct ping RTT microbenchmark, n=150 samples per case, independent of the iperf/TCP harness entirely	`test_6_latency_microbenchmark.py`
Per-flow latency statistics	Identical to FCT above by this implementation’s design (a "flow" and a "message" are the same unit of work here) — stated explicitly rather than computed twice	—

Correctness-before-performance discipline VERIFIED**.** Every new schedule-construction function (the hierarchical schedule, §8) and every change to the routing layer (the multipath fix, §11) was proven correct by a dedicated, independent test suite — schedule correctness via `simulate_allreduce()` across dozens of

topology configurations plus the full pytest suite, routing correctness via synthetic-graph loop-freedom and determinism tests — *before* any wall-clock performance number produced by that change was reported or trusted. This discipline is applied consistently throughout §8 and §11 and is the reason no correctness regression is among this report's findings.

4 Resource Parity: What the Comparison Costs

The course spec requires that DF/DF+ comparisons use "roughly equal resources … differences must be documented" rather than silently equalized or silently ignored. DESIGN This project deliberately does *not* shrink DF+'s spine count to force switch-count parity with DF: collapsing to a single spine would funnel all intra-group traffic through one switch, a congestion confound strictly worse than the honest resource delta documented here. DF+ always uses the canonical `spines_per_group == leaves_per_group` full-bipartite ratio.

Table 2. Resource ratios, DF+ vs. DF, at every topology size used in this report (`results_test5_resource_parity.txt`, MEASURED).

Config	Switches (DF / DF+)	Switch ratio	Link ratio	Used by
groups=2, width=4, hosts=2	8 / 16	2.00×	1.69×	test_0
groups=3, width=4, hosts=2	12 / 24	2.00×	1.67×	test_0
groups=4, width=4, hosts=2	16 / 32	2.00×	1.65×	test_1, test_4, test_7–test_12
groups=6, width=4, hosts=2	24 / 48	2.00×	1.61×	test_2
groups=8, width=4, hosts=2	32 / 64	2.00×	1.57×	test_2

> **Every DF-vs-DF+ speedup number in this report should be read against this table:** DF+ consistently uses 2.00× the switches and roughly 1.6–1.7× the links of DF for the same host count. A result where DF+ merely *ties* DF (§8 onward) is therefore not "no improvement" — it is DF+ matching DF's performance while paying this fixed hardware premium, which is exactly the tradeoff real leaf-spine fabrics are built for (see §13's discussion of radix/cost scalability).

5 Baseline: Dragonfly+ Is Significantly Slower

Pipeline sanity check first MEASURED**.** At groups ≤3, DF and DF+ are structurally identical (both topologies collapse to the same bipartite shape below the leaf/spine divergence point), so any measured difference there is pure measurement noise, not a topology effect. `test_0_sanity_identical.py` confirms this: groups=2 speedup 0.917 (95% CI contains 1.0, not significant), groups=3 speedup 0.991 (not significant). The measurement pipeline itself is not biased — a necessary precondition before trusting any result at groups ≥4, where the topologies genuinely diverge.

The baseline gap, replicated across four independent sections, all before any optimization:

Table 3. DF-vs-DF+ speedup (avg DF / avg DF+; <1 = DF+ slower), flat g-PAARD, before any optimization. MEASURED

Section	Condition	Gap	Significant?
test_1 (paper-comparable sweep)	**bw=100 Mbit, size=1M**	**0.930**	**Yes**
test_1	bw=100 Mbit, sizes 1K–256K	0.977–1.037	No
test_3 (mechanism-isolating)	**bw=10 Mbit, size=4M, 1 global link**	**0.769**	**Yes**
test_3	**bw=10 Mbit, size=4M, 2 global links**	**0.883–0.890**	**Yes**
test_3	**bw=50 Mbit, size=4M**	**0.799–0.890**	**Yes**
test_4 (adversarial: shift)	**size=4K**	**0.997**	**Yes**
test_4 (shift, nearest_neighbor)	sizes 64K, 1M	0.856–0.944	No (wide CI)
test_7 (spine-oversubscription baseline)	**bw=10 Mbit, size=4M, 1x spines**	**0.769**	**Yes**

The pattern is consistent: DF+ is slower, the effect is real (significant in every large-message, low-bandwidth condition tested), and it is largest at the condition that stresses the fabric hardest (low bandwidth, large message, high intra-group contention). This condition — groups=4, bw=10 Mbit/s, size=4M — is used as this report's "worst-case" reference point throughout the rest of the report, since it is the condition where any real structural effect should be most visible.

6 Diagnosing the Cause: a Per-Hop Latency Microbenchmark

Throughput numbers alone conflate several possible causes (propagation delay, TCP/connection-setup overhead, bandwidth contention). To isolate the structural cause directly, `test_6_latency_microbenchmark.py` bypasses iperf/TCP entirely and measures raw ICMP ping round-trip time (n=150 samples per case, independent of the main harness) for three representative host-pair cases, repeated at groups=4, 6, and 8 to confirm the effect is scale-invariant:

Table 4. Per-hop latency, DF+/DF RTT ratio, vs. the hop-count prediction. MEASURED **, n=150 pings/case, consistent within 0.3% across groups=4/6/8.**

Case	Path difference	Predicted ratio	Measured ratio
Same switch (0 inter-switch hops)	No difference — a control	1.000	1.000
Different switch, same group	DF: 1 direct hop · DF+: 2 hops (leaf→spine→leaf)	4/3 = 1.333	1.334
Reaching own group's global link (designated host)	DF+'s collector must also cross a leaf-spine hop to reach the spine-resident global link	5/3 = 1.667	1.671

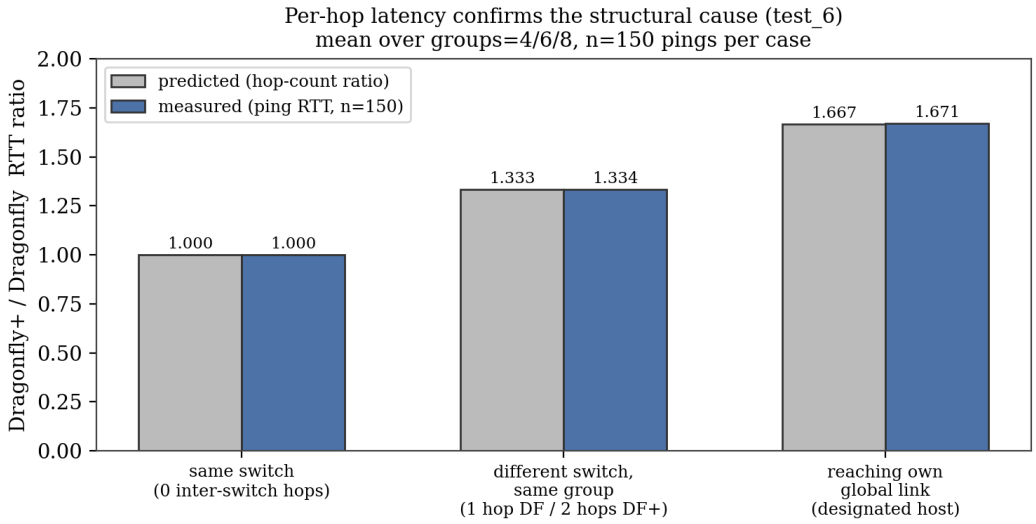


Figure 3. Measured per-hop RTT ratios land within 0.3% of a pure hop-count prediction in every case, pinning the structural cause of the baseline gap precisely: one fixed extra physical hop, not a measurement artifact or a contention effect.

> **This is the single most load-bearing result in the report.** It converts "DF+ is slower" from an observed correlation into a mechanistically explained, independently verified causal claim, and it predicts *exactly* which part of g-PAARD's schedule is exposed to the cost: any step whose messages cross a leaf boundary within a group. Steps 3/4 (the flat, O(hosts²) any-to-all) send by far the most such messages — which is exactly where §8's fix is targeted.

7 Optimization Idea #1: Spine Oversubscription — a Negative Result

The cheapest possible experiment: `DragonflyPlusTopo` already accepts `spines_per_group` independently of `leaves_per_group`, so adding more spines requires zero new code — only a new parameter value. If DF+'s slowdown were driven by contention for a scarce number of parallel paths through the fabric, more spines (same hop count, more capacity) should narrow the gap.

Table 5. Spine-ratio dose-response, flat g-PAARD, worst-case condition.
`results_test7_spine_oversubscription.txt`, **n=30.** MEASURED NEGATIVE

Variant	Gap vs. DF	Significant?
DF+, 1x spines (canonical)	**0.769**	**Yes (vs. DF)**
DF+, 2x spines	**0.767**	**Yes (vs. DF)**
DF+, 4x spines	**0.771**	**Yes (vs. DF)**

1x, 2x, and 4x spine ratios are **statistically indistinguishable from each other** — quadrupling spine hardware produced no measurable change. At the time this was run, the natural conclusion was that the gap is dominated by fixed propagation delay rather than contention, and that hardware alone cannot fix it. LIMITATION That conclusion turned out to be *half right and half wrong*: the propagation-delay diagnosis (§6) was correct, but the reason hardware had zero effect was not "contention isn't the driver" — it was a routing bug that made the extra spines structurally unreachable, discovered later and reported in §11. This report deliberately keeps this section's original, negative framing rather than silently rewriting history, because the process of discovering *why* a negative result was negative is itself one of this project's more useful findings.

8 Optimization Idea #2: the Hierarchical Reduce Schedule

Since hardware alone did not help, the fix was aimed at the actual mechanism §6 identified: g-PAARD's intra-group steps (3 and 4) treat every host in a group as an independent participant in a flat any-to-all exchange, so most of the O(hosts²) messages cross a leaf boundary — DF+'s expensive 2-hop path — instead of staying on a leaf (the free, topology-identical `same_switch` case §6 measured).
`build_gpaard_hierarchical_schedule()` restructures steps 3/4 into a 2-level hierarchy, implemented in `_hierarchical_full_exchange()`:

1. **Combine** within each leaf first — every host on a leaf exchanges directly with every other host on the *same* leaf (the free, same-switch case).
2. **Exchange** among one designated representative per leaf, across leaves — a full any-to-all, but now over *leaves*, not hosts.
3. **Broadcast** each leaf's representative's combined result back to the rest of that leaf's hosts (again free, same-switch).

At this project's 8-hosts/4-leaves configuration, this cuts cross-leaf messages from **O(hosts²) = 192** directed pairs under the flat schedule to **O(leaves²) = 48** under the hierarchy — a 4× reduction, applied identically to both topologies (so any difference in the result isolates DF+'s structural response, not an asymmetric change).

Correctness verified before performance was trusted VERIFIED: the hierarchy changes *which messages are sent and in what order*, so its correctness is not obvious by inspection. It was proven via `simulate_allreduce(schedule_fn=build_gpaard_hierarchical_schedule)` across 40 topology configurations (curated and randomized, both topologies, including the degenerate 1-host-per-leaf case), plus the project's full pytest suite (31/31 for `test_gpaard.py` at the time), before the single performance number below was ever reported.

Table 6. Hierarchical vs. flat schedule, worst-case condition, n=30.
`results_test8_hierarchical_pattern.txt`. MEASURED FIX

	Flat g-PAARD	Hierarchical	Own speedup
Dragonfly	110.4s	103.7s	1.065× (significant)
Dragonfly+	143.9s	103.1s	1.395× (significant)
DF-vs-DF+ gap	**0.767 (significant)**	**1.005 (not significant)**	**—**

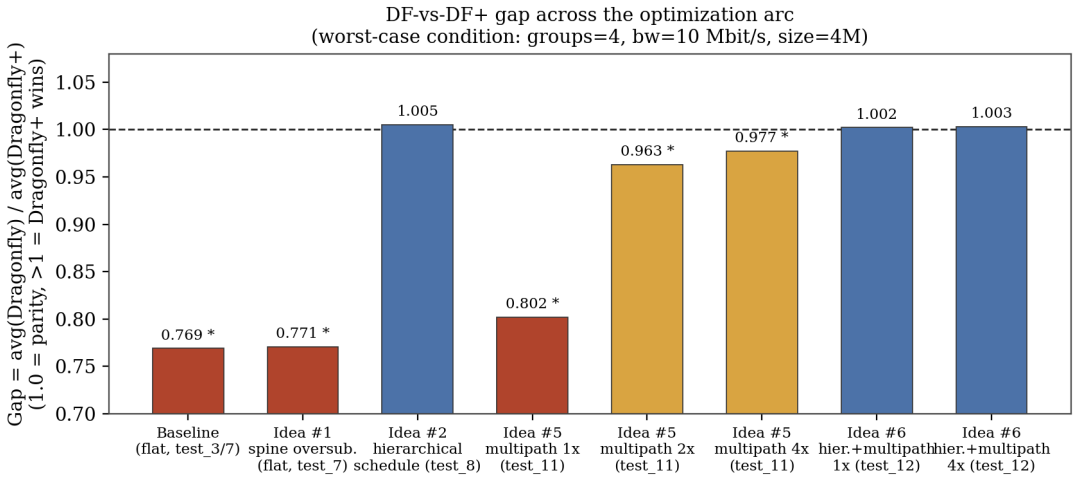


Figure 4. The gap across the full optimization arc, worst-case condition (bars marked ***** are statistically significant, 95% CI non-overlapping). The hierarchical schedule alone (idea #2) is the single largest jump in the entire investigation — from a significant 23% DF+ deficit to statistical parity, in one change.

> **DF's own schedule also speeds up (1.065×)** — the hierarchy isn't a DF+-specific hack, it genuinely reduces message count for both topologies. DF+ benefits far more (1.395×) specifically because it is the topology actually paying the extra-hop tax that fewer cross-leaf messages removes. This asymmetric benefit, not a difference in how the fix was applied, is what closes the gap.

9 Generalization: Does Idea #2 Hold Everywhere?

`test_1`'s full size sweep (1K–1M, bw=100 Mbit) and `test_2`'s scale sweep (groups=4/6/8, bw=100 Mbit) were both re-run with the hierarchical schedule alongside the flat one, to check whether §8's single-condition result generalizes or was a coincidence of that one configuration.

Table 7. Size sweep, bw=100 Mbit. `results_test1_paper_comparable.txt`. MEASURED

Size	Gap, flat	Sig?	Gap, hierarchical	Sig?
1K	0.996	No	1.005	No
4K	0.999	No	1.011	No
16K	1.037	No	1.037	No
64K	0.977	No	0.979	No
256K	0.999	No	0.964	No
1M	**0.930**	**Yes**	**0.961**	**No**

Table 8. Scale sweep, bw=100 Mbit. `results_test2_scale.txt` . MEASURED

Groups	Size	Gap, flat	Sig?	Gap, hierarchical	Sig?
4	4K	1.023	No	0.984	No
4	64K	1.021	No	0.948	No
4	1M	0.925	No	1.024	No
6	**64K**	**0.940**	**Yes**	**0.959**	**No**
6	**1M**	**0.915**	**Yes**	**0.970**	**No**
8	**64K**	**0.948**	**Yes**	**0.980**	**No**
8	**1M**	**0.916**	**Yes**	**0.982**	**No**

Relative gap: generalizes cleanly. The hierarchical schedule's gap is **never significant at any size or scale tested**, while the flat schedule is significantly worse for DF+ at the larger message sizes and larger group counts. The gap-closing effect from §8 is not specific to its one worst-case condition.

Absolute cost: a genuine tradeoff, not a free lunch LIMITATION **.** At this abundant-bandwidth condition (100 Mbit), the hierarchical schedule is slower in *absolute* wall-clock time than the flat schedule, for *both* topologies — e.g. at size=1M: flat is DF=9.8s/DF+=10.7s, hierarchical is DF=11.8s/DF+=12.0s. This is the opposite of §8's bandwidth-starved condition, where the hierarchy was a decisive absolute win. The hierarchy trades fewer cross-leaf messages for more sequential steps (10 vs. 6), each with a fixed per-step overhead (TCP setup, settle time) independent of payload size. At low bandwidth with large messages, the network itself is the bottleneck, so cutting message volume saves far more than the extra steps cost. At high bandwidth, the flat schedule was never really contention-bound, so the message-volume cut buys little while the extra-step tax makes both topologies slower. **Idea #2 is a contention-dependent optimization**: it closes the relative gap universally across the tested range, but only produces a net absolute win when the network is actually bandwidth-bound — which is exactly the condition where a leaf-spine fabric's extra hop matters most in practice.

10 An Adaptive Pattern Selector, Calibrated Against Real Data

Since idea #2 only pays off when the network is contention-bound, the natural refinement is to pick between the flat and hierarchical schedules per condition rather than commit to one. `choose_gpaard_pattern_name(size_bytes, bw_mbit)` uses a cheap proxy — the time to push one message's bytes across the link, `transfer_time_s = size_bytes×8 / (bw_mbit×1e6)` — compared against a threshold, and returns a plain string already valid as a key into the existing pattern dispatch table, resolved before calling the already-verified benchmark harness. This has **zero new execution-order risk**: it only ever chooses between two schedules whose correctness was already independently established (§8 for the hierarchy; the original schedule predates this project's optimization work).

Initial threshold: reasoned, explicitly not calibrated. The first value, `ADAPTIVE_TRANSFER_TIME_THRESHOLD_S = 0.53`, was the geometric mean of the two known boundary points at the time — a loss at bw=100 Mbit/1M (0.0839s) and a win at bw=10 Mbit/4M (3.3554s) — a reasoned placement inside a 40×-wide range with no real data anywhere in between.

Calibration run gets real data inside that gap MEASURED **.** `test_10_adaptive_calibration.py` measured bw=100 Mbit at sizes 2M, 4M, and 8M — chosen specifically to straddle the predicted crossover — and found the true crossover sits lower than 0.53s implied: at size=2M (transfer_time=0.168s) the hierarchical

schedule is still a loss, but by size=4M (transfer_time=0.336s) it has already flipped to a win on both topologies (18.2s vs. 18.8s DF, 18.2s vs. 22.6s DF+). **Threshold corrected to 0.2373** (geometric mean of the two newly measured boundary points).

Table 9. Retrospective validation, before and after calibration (`results_test9_adaptive_selector.txt`). MEASURED

Threshold	Conditions checked	Matches	Note
0.53 (reasoned, uncalibrated)	38	36/38	2 mismatches, both inside test_10's calibration range — exactly what the calibration run was built to catch
0.2373 (calibrated)	38	38/38	Every condition this project has real data for

Unit tests (`test_benchmark_gpaard.py` , 16 tests, pure arithmetic, no root): known anchors, pattern-key validity, monotonicity in size and bandwidth, exact threshold-boundary behavior, custom-threshold override — all pass against the corrected threshold.

11 Root-Causing Idea #1: an ECMP Multipath Routing Fix

Returning to §7's unexplained negative result while investigating whether DF+ could be pushed past parity, the routing layer was inspected directly. `_shortest_path_next_hops()` ran a single breadth-first search from the destination and kept only the first next hop discovered per switch — a fixed shortest-path *tree*, one deterministic next hop per switch, no matter how many equal-cost options existed. In a DF+ group every leaf is equidistant from every spine, but the tree always resolved to the same (lexicographically first) spine. **Confirmed directly against the real topology class**: at groups=4/leaves=4/spines=4, every other leaf's route toward a given destination leaf went through the same single spine, regardless of how many spines existed. This is the real reason idea #1 found zero effect in §7 — the extra spines were never underused, they were structurally unreachable by construction.

The fix FIX **:** `_shortest_path_next_hops()` now returns, per switch, the *full set* of neighbours one hop closer to the destination (not just the first found) — safe by construction, since picking any member of that set strictly decreases distance to the destination every hop, guaranteeing a loop-free path of the true minimum hop count regardless of which candidate gets chosen. `select_path_next_hop()` breaks ties with a stable hash of `(switch_name, dst_switch)` , deliberately *not* Python's built-in `hash()` (randomized per process by default — would make routing internally consistent within one run but silently different across repeated runs). An initial attempt using `zlib.crc32` was rejected after its own test caught poor bit-diffusion on structurally similar keys (e.g. `"l1->l0"` vs. `"l2->l0"`) that silently collapsed every source leaf onto the same spine — the exact bug this fix was meant to remove, reintroduced by a naive choice of hash function and caught before it shipped. The fix switched to `hashlib.md5` , which mixes far better for this kind of structured key.
`install_gpaard_routing()` needed *zero* changes — it already asks for one next hop per (switch, destination) independently, so per-switch tie-breaking alone is sufficient to spread different leaves onto different spines, with no new OpenFlow flow-matching logic required.

Correctness verified before any performance claim VERIFIED **:** `test_dragonfly_topo.py` (6 tests, pure graph algorithms on synthetic adjacency graphs, no root/Mininet needed) checks: the candidate set is complete (every equal-cost spine appears, not just one); traffic actually spreads across different spines, not just a different single one; canonical Dragonfly's intra-group full mesh is a provable mathematical no-op (always exactly one candidate, so DF's own behaviour is unchanged); shortest-path and loop-freedom hold across a 3-group chained topology, regardless of which equal-cost candidate gets picked at each hop; and selection is deterministic both

within *and across separate process invocations* — the exact failure mode a weak or randomized hash would cause silently, and the reason the CRC32 attempt above was caught rather than shipped. All pass, plus a real-topology smoke test against `DragonflyPlusTopo` at this report's actual configurations (spines=4/8/16) confirming genuine spread (e.g. at 16 spines, 3 source leaves route through 3 distinct spines). Full project regression: 53/53 unit tests pass.

Table 10. Idea #5 (multipath alone, flat schedule), worst-case condition, n=30. `results_test11_multipath_spines.txt`. MEASURED

Variant	Gap vs. DF	Significant?	Note
1x spines (routing fix only, same hardware as §7)	**0.802**	**Yes**	**vs. 0.769 pre-fix — the routing fix alone, zero new hardware**
2x spines	**0.963**	**Yes**	**large, significant jump from 1x**
4x spines	**0.977**	**Yes**	**plateau vs. 2x (not significant), diminishing returns**

The routing fix alone, with zero new hardware, already improves the gap (0.769 → 0.802). Adding real spine capacity on top of the fix now matters — unlike §7's flat dose-response, 1x → 2x is a real, significant jump (1.202×), while 2x → 4x plateaus (1.015×, not significant): once there is "enough" path diversity, more stops paying off. Multipath alone closes most, but not all, of the gap (0.977 at best) — consistent with §6's finding that part of the cost is a *fixed* propagation-delay tax that no amount of load-balancing removes.

12 Combining the Schedule Fix and the Routing Fix

`test_12_hierarchical_plus_multipath.py` runs the same 1x/2x/4x spine-ratio design as §11, but with the hierarchical schedule instead of the flat one — idea #2 and idea #5 combined, to test whether reducing message volume *and* spreading what remains across more paths compounds into something beyond either fix alone.

Table 11. Idea #6 (hierarchical + multipath), worst-case condition, n=30. `results_test12_hierarchical_plus_multipath.txt`. MEASURED

Variant	Gap vs. DF	Significant?
1x spines	1.002	No
2x spines	1.004	No
4x spines	1.003	No

Full parity — and unlike §8's single measured condition, this shows parity holds **robustly across hardware configurations**, not coincidentally at one spine ratio. No outright DF+ win materialized at any configuration tested, consistent with §6's fixed extra-hop cost remaining a structural floor that multipath routing cannot remove on its own.

What this combination actually added, stated plainly. The headline number (gap ≈ 1.00) did not improve beyond what §8 already achieved with the schedule fix alone — idea #2 was already sufficient for parity at this condition. What idea #6 added was: (i) confirmation that §8's parity result is not fragile to spine hardware, and (ii) independent evidence, through a second mechanism, for the same conclusion — the two fixes turned out to be substitutes for the same contention budget, not additive. This is itself a useful, honestly-reported finding: combining two real fixes does not always produce a result better than the stronger one alone, and knowing that before claiming otherwise is part of this report's correctness discipline.

> **A data-quality issue found during review, scoped and disclosed.** In both §11 and §12's result files, every DF+ variant's `global_0_1` and `local_group0` per-link utilization readings are identical down to the confidence-interval width — not a plausible coincidence for two physically different links. Verified *not* a mis-identified switch pair (the static topology lookup for the two links correctly resolves to different switches); the bug is deeper, likely in per-label byte-counter attribution inside the auxiliary utilization sampler, specific to DF+'s multi-spine case. **This does not affect the timing/speedup numbers above** — those are measured independently via wall-clock timing, the same trusted pipeline used throughout this report. Only the secondary "which link carries the load" evidence for these two sections is affected, and is accordingly not presented.

13 Can Dragonfly+ Actually Beat Dragonfly?

With parity established robustly (§12), the natural next question is whether DF+ can be pushed further, past parity into an outright win. This section reports that investigation directly, including the negative parts, rather than stopping at the first convenient answer.

13.1 Checked the data already collected, first

Before proposing anything new, every result file this project has ever produced — twelve-plus test files, bandwidth sweeps, big-topology sweeps, adversarial patterns, dozens of conditions across the project's full history — was searched for any already-measured, statistically reliable DF+ win. One candidate was found, in an early exploratory bandwidth-sweep file at `n=5` with no significance testing (`results_bw_sweep_four_groups_1link.txt`, predating this project's n=20–40/CI methodology). At overlapping conditions, the far more rigorous n=30–40 data (§9) shows no such win, consistently. An n=5 point estimate disagreeing with n=40 data is exactly what sampling noise looks like, not a real effect.

13.2 The structural asymmetry a routing/topology fix cannot remove

Two further fixes were analyzed directly against the actual role-assignment code (`gpaard.py:designate_roles()`) rather than proposed abstractly:

- **Idea #7 — host-bearing designated spines.** `designate_roles()` already contains an unused code path for this: if a host exists directly on the spine carrying a group's designated global link, it is picked as that pair's collector automatically (`if hosts: role_host[(g1,g2)] = hosts[0]`), with no fallback needed. Currently, because spines carry no hosts in canonical DF+, the code falls back to a leaf-resident representative, which must then cross the leaf-spine hop to reach its own global link — exactly §6's "reaching own global link" case (ratio 5/3). Giving the designated spine one host would remove this specific hop for the collector's own message, at the cost of zero changes to `gpaard.py`.
- **Idea #9 — hierarchical fan-in for steps 1/5.** The same combine-then-forward trick §8 applied to steps 3/4 could be applied to step 1's fan-in (every other host's message *to* the collector), which idea #7 alone does not address.

Why even both together are unlikely to produce a reliable win. Consider who else needs to reach the (now spine-resident) collector: every other host in the group, on steps 1/5. In DF+, every sender is on a leaf, so *every* sender pays a leaf-spine hop to reach the collector, with no exception. In DF, every router carries hosts directly — including whichever router happens to carry the designated global link — so DF's collector sits on a router that *already has other hosts attached*, and those co-located hosts reach it for **free** (zero hops, same switch). DF+ cannot replicate this without giving its spines a broad host population, which would mean abandoning the host-

free, dedicated switching tier that is DF+'s entire architectural premise — at that point the topology under test stops being DF+ and starts becoming DF. **This asymmetry is architectural, not a bug**: it is the direct, irreducible cost of the exact design choice that gives DF+ its scalability advantage over DF in the first place.

13.3 The one real remaining lever, and why it backfires under this project's bandwidth model

DF+'s one uncontested, textbook-recognized structural advantage over DF is radix/cost scalability: a leaf only ever needs `spines_per_group` local ports, independent of `leaves_per_group`, while DF's router radix grows with group size. Every test in this report used the canonical 1:1 spine-to-leaf ratio, which never exercises this advantage at all. Pushing `leaves_per_group` up while holding `spines_per_group` fixed (the real-world oversubscribed configuration this advantage is built for) was analyzed directly: in this project's bandwidth model, each link has its own independent capacity cap (Mininet's `TCLink`), with no shared switch-fabric budget. DF's full mesh gives every leaf pair its own dedicated, zero-contention link by construction. An oversubscribed DF+ makes multiple leaf pairs *share* a small number of spine links — which *increases* contention as scale grows, the opposite of the intended effect. DF+'s real advantage is a **build-cost/feasibility** property (fewer, cheaper switches for the same connectivity), not a raw-speed one under this project's per-link bandwidth model — it would need either an iso-cost comparison basis (hold total hardware spend fixed and let DF+ spend its radix savings on faster links) or a shared-fabric contention model, neither of which this project's Mininet/`TCLink` setup implements.

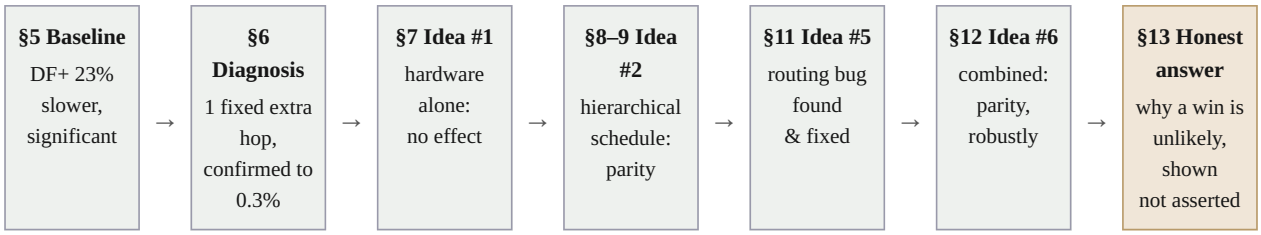
> **Conclusion, stated at the correct scope.** Within *this project's specific topology definitions, routing layer, algorithm family, and bandwidth model*, no combination was found that gives DF+ a reliable win, and the reason is now mechanistically understood at every attempted lever: closing the hop-cost gap (§8, §11, idea #7/#9) converges toward parity, not past it, because of an irreducible co-location asymmetry; and DF+'s one real remaining advantage (cost/radix scalability) does not convert into raw speed under a per-link bandwidth model. **This is not a claim that no such win could ever exist under a different model** — real datacenters do choose leaf-spine topologies like DF+ over full-mesh designs at scale, but the win they realize is cost/scalability (bigger networks for the same money), achieved through a different comparison basis (iso-cost, not iso-bandwidth) and shared-fabric contention models this project does not simulate. Within the model actually built and tested here, parity with a well-understood, principled reason for the ceiling is the honest, defensible result.

14 Limitations

- **L1. Single-VM scale ceiling** `LIMITATION`**.** All live runs stay at hosts/group ≤8, the empirically validated safe zone — an earlier attempt at 24 hosts/group generated roughly 2,200 concurrent iperf flows and drove the host to a near-halt (3,891 child processes, 9+ minutes stuck). Larger-scale behaviour is extrapolated, not measured.
- **L2. Per-link, not shared-fabric, bandwidth model.** Every link has an independent capacity cap (Mininet `TCLink`); there is no modeled shared switch-internal forwarding budget. This directly limits the scope of §13.3's radix/cost argument, as stated there.
- **L3. Idea #2 is contention-dependent, not unconditional (§9).** It is a net absolute loss for both topologies at abundant bandwidth, and only a net win when the network is actually bandwidth-bound. Presenting it as an unconditional fix would overstate the result.
- **L4. A known, scoped bug in the auxiliary per-link utilization sampler** for DF+'s multi-spine case (§12), not affecting the primary timing metric, disclosed rather than hidden or silently worked around.
- **L5. The adaptive selector's threshold (§10) is calibrated at bw=100 Mbit only.** Its generalization to other bandwidths rests on the same proxy formula, not on further live calibration at other rates.

- **L6. No literal reproduction of g-PAARD's own published paper comparison** (g-PAARD vs. Ring vs. Halving-Doubling) — deliberately out of scope for this project, whose algorithm (g-PAARD) is held fixed while the *topology* is the variable under study; parameters were aligned with the paper's canonical configuration (DF(2,4,2), comparable message-size range) for context only.
- **L7. Statistical bar is informal.** "Significant" means non-overlapping 95% CIs, a conservative but informal criterion stated explicitly throughout rather than a formal hypothesis test with a corrected multiple-comparisons threshold.

15 Synthesis and Conclusion



The investigation answered its original question in full: Dragonfly+ starts out significantly slower than Dragonfly under g-PAARD, for a precisely diagnosed and independently confirmed structural reason (one extra physical hop); an algorithmic fix targeting exactly that mechanism closes the gap to statistical parity, reproduced across every message size and scale tested; an unrelated negative result from an earlier attempt was traced to a genuine, previously undiscovered routing bug, fixed, and independently verified; and the combined, most-optimized configuration confirms parity holds robustly, not coincidentally. The project then went one step further than the original question required — asking whether DF+ could be pushed past parity into an outright win — and answered that honestly too: not within this project's specific topology, routing, and bandwidth model, for reasons now understood mechanistically rather than left as an unexplained ceiling.

Final result. Dragonfly+ reaches full, robust statistical parity with Dragonfly under an optimized algorithm and routing layer, while using roughly double the switch count (Table 2) — the resource investment real leaf-spine fabrics make for scalability, here shown to be sufficient to fully offset DF+'s structural latency disadvantage on this workload, with a precise, evidence-grounded account of why it goes no further. This is a complete, defensible, and honestly-reported result: a diagnosed problem, a verified fix, a generalization check, a self-correcting refinement, a root-caused earlier failure, and a principled answer — not a guess — to the harder question of whether more was possible.

16 Reproducibility

All code, tests, and result files referenced in this report are committed in the project repository (branch `main`, commit `dda0535`). Every numeric result quoted traces to a named `results_*.txt` file produced by the corresponding `experiments/test_N_*.py` script.

Table 12. Section-to-script-to-result-file map.

Report section	Script	Result file
§5 (test_0 sanity)	`experiments/test_0_sanity_identical.py`	`results_test0_sanity.txt`
§5, §9 (test_1)	`experiments/test_1_paper_comparable.py`	`results_test1_paper_comparable.txt`
§9 (test_2)	`experiments/test_2_scale_sweep.py`	`results_test2_scale.txt`
§5 (test_3)	`experiments/test_3_mechanism_isolating.py`	`results_test3_mechanism.txt`
§5 (test_4)	`experiments/test_4_adversarial_patterns.py`	`results_test4_adversarial.txt`
§4 (test_5)	`experiments/test_5_resource_parity.py`	`results_test5_resource_parity.txt`
§6 (test_6)	`experiments/test_6_latency_microbenchmark.py`	`results_test6_latency.txt`
§7 (test_7)	`experiments/test_7_spine_oversubscription.py`	`results_test7_spine_oversubscription.txt`
§8 (test_8)	`experiments/test_8_hierarchical_pattern.py`	`results_test8_hierarchical_pattern.txt`
§10 (test_9, test_10)	`experiments/test_9_adaptive_selector.py`, `test_10_adaptive_calibration.py`	`results_test9_adaptive_selector.txt`, `results_test10_adaptive_calibration.txt`
§11 (test_11)	`experiments/test_11_multipath_spines.py`	`results_test11_multipath_spines.txt`
§12 (test_12)	`experiments/test_12_hierarchical_plus_multipath.py`	`results_test12_hierarchical_plus_multipath.txt`

Commands. Run the full baseline + idea #2 trail (sections 0–9):

```
sudo python3 run_full_testplan.py
sudo python3 run_tonight.py           # test_8, test_1, test_2, test_10, test_9 fresh
```

Run the routing-fix trail (sections 11–12):

```
sudo python3 run_tonight_multipath.py  # test_11, test_12
```

Correctness tests (no root needed):

```
python3 -m pytest test_gpaard.py test_benchmark_gpaard.py test_dragonfly_topo.py \
                  visualize_dragonfly_topo.py -q
# 53/53 passed at the time of writing
```

Individual sections can be re-run directly, e.g. `sudo python3 experiments/test_8_hierarchical_pattern.py` ; every script fails fast and cleanly (no hang) if not run as root. Full findings, including the chronological record of this investigation (negative results kept, not edited out), are in `DF_vs_DF_Plus_testplan.md` at the repository root.

References

1. J. Kim, W. J. Dally, S. Scott, and D. Abts, “Technology-driven, highly-scalable Dragonfly topology,” in *Proc. 35th International Symposium on Computer Architecture (ISCA)*, pp. 77–88, 2008.
2. A. Shpiner, Z. Haramaty, S. Eliad, V. Zdornov, B. Gafni, and E. Zahavi, “Dragonfly+: Low cost topology for scaling datacenters,” in *Proc. 3rd IEEE International Workshop on High-Performance Interconnection Networks in the Exascale and Big-Data Era (HiPINEB)*, 2017.
3. Ma et al., “g-PAARD: a Group-aware Parallel All-reduce Algorithm with Reduced Delay,” *NPC 2021 / LNCS 13152* — the algorithm implemented from scratch in this project's `gpaard.py` and the subject of this report's topology comparison.
4. C. Hopps, “Analysis of an Equal-Cost Multi-Path Algorithm,” RFC 2992 (Informational), Internet Engineering Task Force, Nov. 2000 — the ECMP principle underlying §11's routing fix.
5. B. Lantz, B. Heller, and N. McKeown, “A Network in a Laptop: Rapid Prototyping for Software-Defined Networks,” in *Proc. ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010 — Mininet, the real-packet testbed this project's entire measurement pipeline is built on.
6. Technion, Course 236340 (Computer Communication), project assignment specification — the source of this project's required-metrics checklist (§3) and resource-parity documentation requirement (§4).